0. Database and table setup

```
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT CHECK (age >= 17),
    city VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    title VARCHAR(80) NOT NULL,
    credits INT DEFAULT 3,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    marks INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Insert the sample data

```
INSERT INTO departments VALUES
(1,'CSE'),(2,'ECE'),(3,'Mechanical'),(4,'Civil');

INSERT   INTO   students   VALUES
(101,'Aarav',20,'Roorkee',1),
(102,'Diya',21,'Dehradun',1),
(103,'Kabir',19,'Haridwar',2),
(104,'Meera',22,'Roorkee',1),
(105,'Rohan',20,'Delhi',3),
(106,'Sana',21,'Dehradun',2),
(107,'Vikram',23,'Jaipur',NULL),
(108,'Isha',19,'Roorkee',2);

INSERT INTO courses VALUES
(201,'Data Structures',4,1),
(202,'Operating Systems',4,1),
(203,'Digital Electronics',3,2),
(204,'Thermodynamics',3,3),
(205,'Machine Learning',4,1);

INSERT    INTO    enrollments    VALUES
(101,201,88),(101,202,76),(101,205,91),
(102,201,67),(102,205,72),
(103,203,81),              (104,201,95),
(104,202,89), (105,204,58),
(106,203,74),
(108,203,66);


SELECT * FROM students;
```

Check: the SELECT statement should return 8 student rows.

Part A — DDL (Data Definition Language)

A1. Create the faculty table

```
CREATE TABLE faculty (
    faculty_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
```

```
    email VARCHAR(100) UNIQUE,
    salary DECIMAL(10,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

A2. Add phone column

```
ALTER TABLE faculty ADD COLUMN phone VARCHAR(15);
```

A3. Rename phone to contact_no

```
ALTER TABLE faculty RENAME COLUMN phone TO contact_no;
```

A4. Drop contact_no

```
ALTER TABLE faculty DROP COLUMN contact_no;
```

A5. Drop faculty and explain DROP, TRUNCATE and DELETE

```
DROP TABLE faculty;

-- DROP: removes the table structure and its data.
-- TRUNCATE: removes all rows but keeps the table structure.
-- DELETE: removes selected rows (with WHERE) or all rows (without WHERE);
--         the table structure remains.
```

Part B — DML (Data Manipulation Language)

B1. Insert Neha

```
INSERT INTO students VALUES (109, 'Neha', 20, 'Roorkee', 1);
```

B2. Insert a student without specifying city

```
INSERT INTO students (student_id, name, age, dept_id)
VALUES (110, 'Aman', 20, 1);

SELECT student_id, name, city
FROM students
WHERE student_id = 110;
```

B3. Update student 105's city

```
UPDATE students SET city = 'Roorkee' WHERE student_id = 105;
```

B4. Increase marks by 5 for course 203

```
UPDATE enrollments SET marks = marks + 5 WHERE course_id = 203;
```

B5. Delete student 109

```
DELETE FROM students WHERE student_id = 109;
```

B6. Try to insert age 15

```
INSERT INTO students (student_id, name, age, city, dept_id)
VALUES (111, 'TestStudent', 15, 'Roorkee', 1);
```

> Expected result: the insert is rejected by CHECK (age >= 17). The wording of the error depends on the database. MySQL commonly reports a check-constraint violation. If the database/version does not enforce CHECK constraints, verify its version and behavior.

Part C — Querying with SELECT

C1. Display all columns of all students

```
SELECT * FROM students;
```

C2. Display only name and city

```
SELECT name, city FROM students;
```

C3. Students from Roorkee

```
SELECT * FROM students WHERE city = 'Roorkee';
```

C4. Students aged between 20 and 22

```
SELECT * FROM students WHERE age BETWEEN 20 AND 22;
```

C5. Students from Roorkee or Dehradun

```
SELECT * FROMstudentsWHERE city IN ('Roorkee', 'Dehradun');
```

C6. Names starting with A

```
SELECT * FROM students WHERE name LIKE 'A%';
```

C7. Students without a department

```
SELECT * FROM students WHERE dept_id IS NULL;
```

C8. Distinct cities

```
SELECT DISTINCT city FROM students;
```

C9. Age descending, then name ascending

```
SELECT * FROM students ORDER BY age DESC, name ASC;
```

C10. Top 3 oldest students

```
SELECT * FROMstudentsORDER   BY age DESC, name ASC LIMIT 3;
```

C11. Aliases Student Name and Age

```
SELECT name AS`StudentName`, age AS Age FROM students;
```

C10 uses name ascending to make the result deterministic when students have the same age. With the original data, the top three are Vikram (23), Meera (22), and Diya (21), because Diya comes before Sana alphabetically.

Part D — Functions and Aggregation

D1. Total number of students

```
SELECT COUNT(*)AStotal_students   FROM students;
```

Expected on the original sample data: 8.

D2. Average, minimum and maximum marks

```
SELECT AVG(marks)AS  average_marks, MIN(marks) AS minimum_marks, MAX(marks) AS maximum_marks FROM enrollments;
```

Original sample data: average ≈ 77.91, minimum = 58, maximum = 95.

D3. Number of students in each city

```
SELECT city, COUNT(*)AS  student_count FROM students GROUP BY city;
```

D4. Average marks for each course

```
SELECT c.course_id,c.title,  AVG(e.marks) AS average_marks
FROM courses c
LEFT JOIN enrollments e ON e.course_id = c.course_id
GROUP BY c.course_id, c.title
ORDER BY c.course_id;
```

Original sample averages: Data Structures 83.33; Operating Systems 82.50; Digital Electronics 73.67; Thermodynamics 58.00; Machine Learning 81.50.

D5. Cities with more than one student

```
SELECT city, COUNT(*)AS  student_count
FROM students
```

```
GROUP BY city
HAVING COUNT(*) > 1;
```

On the original data, Roorkee has 3 students and Dehradun has 2.

D6. Total credits offered by each department

```
SELECT d.dept_id, d.dept_name, COALESCE(SUM(c.credits), 0) AS total_credits
FROM departments d
LEFT JOIN courses c ON c.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY d.dept_id;
```

Original data: CSE = 12, ECE = 3, Mechanical = 3, Civil = 0.

D7. Names in uppercase and name length

```
SELECT UPPER(name) AS uppercase_name, LENGTH(name) AS name_length FROM students;
```

D8. Why WHERE cannot filter aggregates; role of HAVING

WHERE filters individual rows before grouping and aggregation. An aggregate such as COUNT(*) or AVG(marks) is calculated after rows are grouped, so it normally cannot be used in WHERE. HAVING filters the groups after GROUP BY, for example HAVING COUNT(*) > 1.

Part E — Joins

E1. Student name with department name (INNER JOIN)

```
SELECT s.name, d.dept_name
FROM students s
INNER JOIN departments d ON s.dept_id = d.dept_id;
```

E2. All students, including those without a department (LEFT JOIN)

```
SELECT s.name, s.dept_id, d.dept_name
FROM students s
LEFT JOIN departments d ON s.dept_id = d.dept_id;
```

E3. All departments, including those with no students

```
SELECT d.dept_id, d.dept_name, s.name
FROM students s
RIGHT JOIN departments d ON s.dept_id = d.dept_id
ORDER BY d.dept_id, s.name;
```

E4. Student name, course title and marks

```
SELECT s.name, c.title, e.marks
FROM students s
JOIN enrollments e ON e.student_id = s.student_id
JOIN courses c ON c.course_id = e.course_id
ORDER BY s.name, c.title;
```

E5. Students enrolled in Data Structures

```
SELECT DISTINCT s.name
FROM students s
JOIN enrollments e ON e.student_id = s.student_id
JOIN courses c ON c.course_id = e.course_id
WHERE c.title = 'Data Structures';
```

E6. Students not enrolled in any course

```
SELECT s.student_id, s.name
FROM students s
LEFT JOIN enrollments e ON e.student_id = s.student_id
WHERE e.course_id IS NULL;
```

E7. Number enrolled in each course, including zero

```
SELECT c.course_id, c.title, COUNT(e.student_id) AS enrollment_count
FROM courses c
LEFT JOIN enrollments e ON e.course_id = c.course_id
GROUP BY c.course_id, c.title
ORDER BY c.course_id;
```

E8. Each department's student count and average age

```
SELECT d.dept_name,
       COUNT(s.student_id) AS student_count,
       AVG(s.age) AS average_age
FROM departments d
LEFT JOIN students s ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY d.dept_id;
```

E2: Vikram has NULL for dept_id and dept_name. E3: Civil appears even though it has no students. E6: Vikram is the only student not enrolled in any course in the original data. E7 original counts: Data Structures 3, Operating Systems 2, Digital Electronics 3, Thermodynamics 1, Machine Learning 2.

Part F — Subqueries

F1.Studentsolderthantheaverage age

```
SELECT student_id, name, age
FROM students
WHERE age > (SELECT AVG(age) FROM students);
```

F2. Student(s) with the highest marks in any course

```
SELECT s.name, c.title, e.marks
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
JOIN courses c ON c.course_id = e.course_id
WHERE e.marks = (SELECT MAX(marks) FROM enrollments);
```

F3. Courses with at least one enrollment (IN)

```
SELECT course_id, title
FROM courses
WHERE course_id IN (SELECT course_id FROM enrollments);
```

F4. Students enrolled in more than 2 courses

```
SELECT s.student_id, s.name
FROM students s
WHERE s.student_id IN (
    SELECT    student_id
    FROM     enrollments
    GROUP BY student_id
    HAVING COUNT(*) > 2
);
```

F5. Department(s) with the highest number of students (ties included)

```
SELECT d.dept_id, d.dept_name, COUNT(s.student_id) AS student_count
FROM departments d
LEFT JOIN students s ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(s.student_id) = (
    SELECT MAX(dept_count)
    FROM (
        SELECT COUNT(*) AS dept_count
        FROM students
        WHERE dept_id IS NOT NULL
        GROUP BY dept_id
    ) AS counts_by_dept
);
```

F6. For each course, students scoring above that course's average

```
SELECT s.name, c.title, e.marks
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
JOIN courses c ON c.course_id = e.course_id
WHERE e.marks > (
    SELECT AVG(e2.marks)
    FROM enrollments e2
    WHERE e2.course_id = e.course_id
)
ORDER BY c.course_id, e.marks DESC;
```

Original-data checks: F1 returns Diya, Meera, Sana and Vikram. F2 returns Meera with 95 in Data Structures. F4 returns Aarav. F5 returns both CSE and ECE (3 students each). F6 returns Aarav and Meera for Data Structures; Meera for Operating Systems; Kabir for Digital Electronics; and Aarav for Machine Learning.

Part G — Indexes, Views and Transactions

G1. Create an index and inspect the query plan

```
CREATE INDEX idx_students_city ON students(city);

EXPLAIN
SELECT * FROM students
WHERE city = 'Roorkee';
```

EXPLAIN shows the optimizer's planned access method. The index makes indexed lookup possible, but the optimizer may still choose a table scan for a small table. The exact plan depends on the database, statistics and table size.

G2. Create the student_marks view

```
CREATE VIEW student_marksAS
SELECT s.name AS student_name, c.title AS course_title, e.marks
FROM students s
JOIN enrollments e ON e.student_id = s.student_id
JOIN courses c ON c.course_id = e.course_id;
```

G3. Query students scoring above 80

```
 SELECT * FROM student_marks WHERE marks > 80;
```

G4. Drop the view and index

```
DROP VIEW student_marks;
DROP INDEX idx_students_city ON students;
```

PostgreSQL syntax for dropping the index is DROP INDEX idx_students_city; (without ON students).

G5. Update inside a transaction, then ROLLBACK

```
START TRANSACTION;
UPDATE enrollments
SET marks = 100
WHERE student_id = 101 AND course_id = 201;
ROLLBACK;

SELECT marks
FROM enrollments
WHERE student_id = 101 AND course_id = 201;
```

The final SELECT should show 88 if the original sample data was loaded and the update was rolled back.

G6. Repeat the update and COMMIT

```
START TRANSACTION;
UPDATE enrollments
SET marks = 100
WHERE student_id = 101 AND course_id = 201;
COMMIT;

SELECT marks
FROM enrollments
WHERE student_id = 101 AND course_id = 201;
```

The final SELECT should show 100. This change is permanent after COMMIT. Reload/reset the sample data if you need to restore the original value of 88.

Part H — Optional Challenge Problems

H1.Rankstudentsbytotalmarks,includingstudents with no enrollments

```
SELECT student_id, name, total_marks,
       DENSE_RANK() OVER (ORDER BY total_marks DESC) AS marks_rank
FROM (
    SELECT s.student_id, s.name, COALESCE(SUM(e.marks), 0) AS total_marks
    FROM students s
    LEFT JOIN enrollments e ON e.student_id = s.student_id
    GROUP BY s.student_id, s.name
) AS totals
ORDER BY marks_rank, name;
```

With the original data: Aarav 255 (rank 1), Meera 184 (rank 2), Diya 139 (rank 3), Kabir 81 (rank 4), Sana 74 (rank 5), Isha 66 (rank 6), Rohan 58 (rank 7), Vikram 0 (rank 8).

H2. Assign a grade to every enrollment using CASE

```
SELECT student_id,course_id,  marks,
       CASE
       WHEN marks >= 90 THEN 'A'
       WHEN marks BETWEEN 75 AND 89 THEN 'B'
       WHEN marks BETWEEN 60 AND 74 THEN 'C'
       ELSE 'D'
       END AS grade
FROM enrollments
ORDER BY student_id, course_id;
```

H3. Find pairs of students from the same city (self join)

```
SELECT  s1.name  ASstudent_1, s2.name AS student_2, s1.city
FROM students s1
JOIN students s2
  ON s1.city = s2.city
 AND s1.student_id < s2.student_id
ORDER BY s1.city, s1.name, s2.name;
```

The condition s1.student_id < s2.student_id prevents a student from pairing with themself and avoids showing each pair twice. NULL city values would not match each other.

H4. Find the second-highest distinct mark without LIMIT

```
SELECT MAX(marks)AS  second_highest_marks
FROM enrollments
WHERE marks < (SELECT MAX(marks) FROM enrollments);
```

For the original sample data, the second-highest distinct mark is 91.

Final checklist

- Run thesetupand insertsonly once on a fresh database.
- Verify that SELECT * FROM students returns 8 rows before adding test data.
- Use the original sample data for expected-result checks; B4 and G6 intentionally modify data.
- For a PostgreSQL lab, replace USE college_db with a connection command in psql and adjust DROP INDEX syntax.
- Save screenshots of query results and errors if your instructor asks for evidence.

EXPECTED OUTPUT TABLES

These are illustrative result tablesbasedontheoriginalsampledata.Resultscandifferafterrunningdata-changing tasks such as B4, B5, and G6.

C1. All students

C1. All students

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Delhi	3
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	NULL
108	Isha	19	Roorkee	2

C2. Student names and

name	city
Aarav	Roorkee
Diya	Dehradun
Kabir	Haridwar
Meera	Roorkee
Rohan	Delhi
Sana	Dehradun
Vikram	Jaipur
Isha	Roorkee

C3. Students from Roorkee

C3. Students from Roorkee

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
104	Meera	22	Roorkee	1
108	Isha	19	Roorkee	2

C4. Age between 20 and 22

student_id	name	age
101	Aarav	20
102	Diya	21
104	Meera	22
105	Rohan	20
106	Sana	21

C5. Students from Roor

name	city
Aarav	Roorkee
Diya	Dehradun
Meera	Roorkee
Sana	Dehradun
Isha	Roorkee

C6. Names starting with

student_id	name
101	Aarav

C7. Students without a department

student_id	name	dept_id
107	Vikram	NULL

C8. Distinct

city
Dehradun
Delhi
Haridwar
Jaipur
Roorkee

C9. Age descending, na

name	age
Vikram	23
Meera	22
Diya	21
Sana	21
Aarav	20
Rohan	20
Isha	19
Kabir	19

C10. Top 3 oldest stude

name	age
Vikram	23
Meera	22
Diya	21

C11. Aliased name and ag

Student Name	Age
Aarav	20
Diya	21
Kabir	19
Meera	22
Rohan	20
Sana	21
Vikram	23
Isha	19

D1. Total numbe

total_students
8

D2. Mark summary

average_marks	minimum_mark	maximum_mark
77.91	58	95

D3. Students per city

city	student_count
Dehradun	2
Delhi	1
Haridwar	1
Jaipur	1
Roorkee	3

D4. Average marks per course

course	average_marks
Data Structures	83.33
Operating Systems	82.5
Digital Electronics	73.67
Thermodynamics	58.0
Machine Learning	81.5

D5. Cities with more than o

city	student_count
Dehradun	2
Roorkee	3

D6. Total credits per depart

department	total_credits
CSE	12
ECE	3
Mechanical	3
Civil	0

D7. Uppercase names and nam

uppercase_name	name_length
AARAV	5
DIYA	4
KABIR	5
MEERA	5
ROHAN	5
SANA	4
VIKRAM	6
ISHA	4

E1. Students with depar

name	dept_name
Aarav	CSE
Diya	CSE
Kabir	ECE
Meera	CSE
Rohan	Mechanical
Sana	ECE
Isha	ECE

E2. All students and dep

name	dept_name
Aarav	CSE
Diya	CSE
Kabir	ECE
Meera	CSE
Rohan	Mechanical
Sana	ECE
Vikram	NULL
Isha	ECE

E3. All departments, includ

dept_name	student_name
CSE	Aarav
CSE	Diya
CSE	Meera
ECE	Kabir
ECE	Sana
ECE	Isha
Mechanical	Rohan
Civil	NULL

E4. Student, course and marks

student_name	course_title	marks
Aarav	Data Structures	88
Aarav	Machine Learning	91
Aarav	Operating Systems	76
Diya	Data Structures	67
Diya	Machine Learning	72
Isha	Digital Electronics	66
Kabir	Digital Electronics	81
Meera	Data Structures	95
Meera	Operating Systems	89
Rohan	Thermodynamics	58
Sana	Digital Electronics	74

E5. Student

name
Aarav
Diya
Meera

E6. Students not enrolle

student_id	name
107	Vikram

E7. Enrollments per course

course_title	enrollment_count
Data Structures	3
Operating Systems	2
Digital Electronics	3
Thermodynamics	1
Machine Learning	2

E8. Student count and average age by de

department	student_count	average_age
CSE	3	21.0
ECE	3	19.67
Mechanical	1	20.0
Civil	0	NULL

F1. Students older thar

name	age
Diya	21
Meera	22
Sana	21
Vikram	23

F2. Highest mark in any course

student_name	course_title	marks
Meera	Data Structures	95

F3. Courses with at least one enr

course_id	title
201	Data Structures
202	Operating Systems
203	Digital Electronics
204	Thermodynamics
205	Machine Learning

F4. Students enrolled in more than 2 c

student_id	name	course_count
101	Aarav	3

F5. Department(s) with mos

department	student_count
CSE	3
ECE	3

F6. Students above their course average

student_name	course_title	marks
Aarav	Data Structures	88
Meera	Data Structures	95
Meera	Operating Systems	89
Kabir	Digital Electronics	81
Sana	Digital Electronics	74
Aarav	Machine Learning	91

G3. View rows with marks above 80

student_name	course_title	marks
Aarav	Data Structures	88
Aarav	Machine Learning	91
Kabir	Digital Electronics	81
Meera	Data Structures	95
Meera	Operating Systems	89

G5. Mark after ROLLBACK

student_id	course_id	marks
101	201	88

Expected value when starting from the origin

G6. Mark after COMMIT

student_id	course_id	marks
101	201	100

This reflects the intentional committed updat

H1. Rank by total marks

name	total_marks	rank
Aarav	255	1
Meera	184	2
Diya	139	3
Kabir	81	4
Sana	74	5
Isha	66	6

H2. Grades for each enrollment

student_id	course_id	marks	grade
101	201	88	B
101	202	76	B
101	205	91	A
102	201	67	C
102	205	72	C
103	203	81	B
104	201	95	A
104	202	89	B
105	204	58	D
106	203	74	C
108	203	66	C

H3. Pairs of students from the same

student_1	student_2	city
Aarav	Meera	Roorkee
Aarav	Isha	Roorkee
Diya	Sana	Dehradun
Meera	Isha	Roorkee

H4. Second-highest di

second_highest_marks
91